

SupportOps AI

From a support question
to a verified investigation.

A source-grounded design for an enterprise support copilot with persistent workflows, scoped tools, semantic retrieval, and human-approved escalation.

01 / INVESTIGATE

Inspect account diagnostics and relevant runbooks.

02 / VERIFY

Check citations, safety and semantic support.

03 / ACT

Prepare an escalation; execute only after approval.

Document scope

This TDD describes the current repository and a practical path to production. It is based on source revision **603a7da** and the verification record reviewed on **3 October 2026**. Code is the authority where older project notes differ.

Reading guide	Pages
01 System Design	2-10
02 Tech Stack	11-13
03 Product Features	14-15
04 Scalability & Security	16-21

Status vocabulary

IMPLEMENTED = present in code. **LOCAL CONFIG** = enabled in the current development setup. **PROPOSED** = design work required before deployment; not a delivered capability.

Current local setup: live LLM calls, local MiniLM embeddings, demo authentication, and synthetic customer diagnostics/tickets. Production infrastructure and external OIDC deployment have not been validated.

01 / Architecture & boundaries

A modular monolith is the right starting point for this workload.

Support engineers currently have to correlate failed jobs, credential permissions, service incidents, support plans and runbooks. SupportOps turns that investigation into an auditable case with evidence-backed findings and, when requested, a reviewable escalation draft.

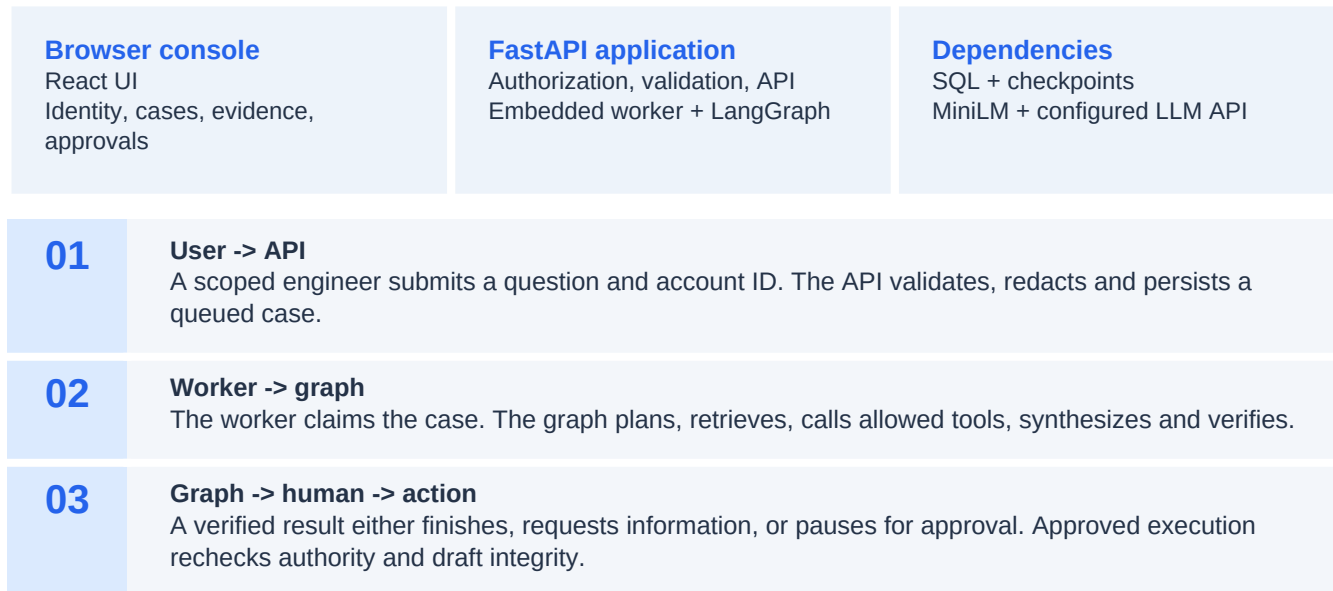


Figure 1. Current logical architecture. API, workflow, retrieval and action modules share one application deployment. The LLM provider is an external trust boundary; local embeddings do not make live inference fully local.

Why not microservices yet?

The system has one cohesive business transaction: investigate and optionally escalate. A modular monolith keeps authorization, action receipts and audit writes close to the data and avoids premature distributed transactions. API handlers, orchestration, model routing, retrieval, security and actions already have separate modules.

Extraction strategy / PROPOSED

Separate API and worker deployments first when concurrency or failure isolation requires it. Extract ingestion/embedding services only when corpus growth or CPU contention justifies independent scaling. Keep policy enforcement at every boundary; a new service must not become a bypass.

Source: backend/supportops/{api,worker,workflow,retrieval,actions}.py

Domain & persistence model

Application relationships are logical today; they are not all database-enforced foreign keys.

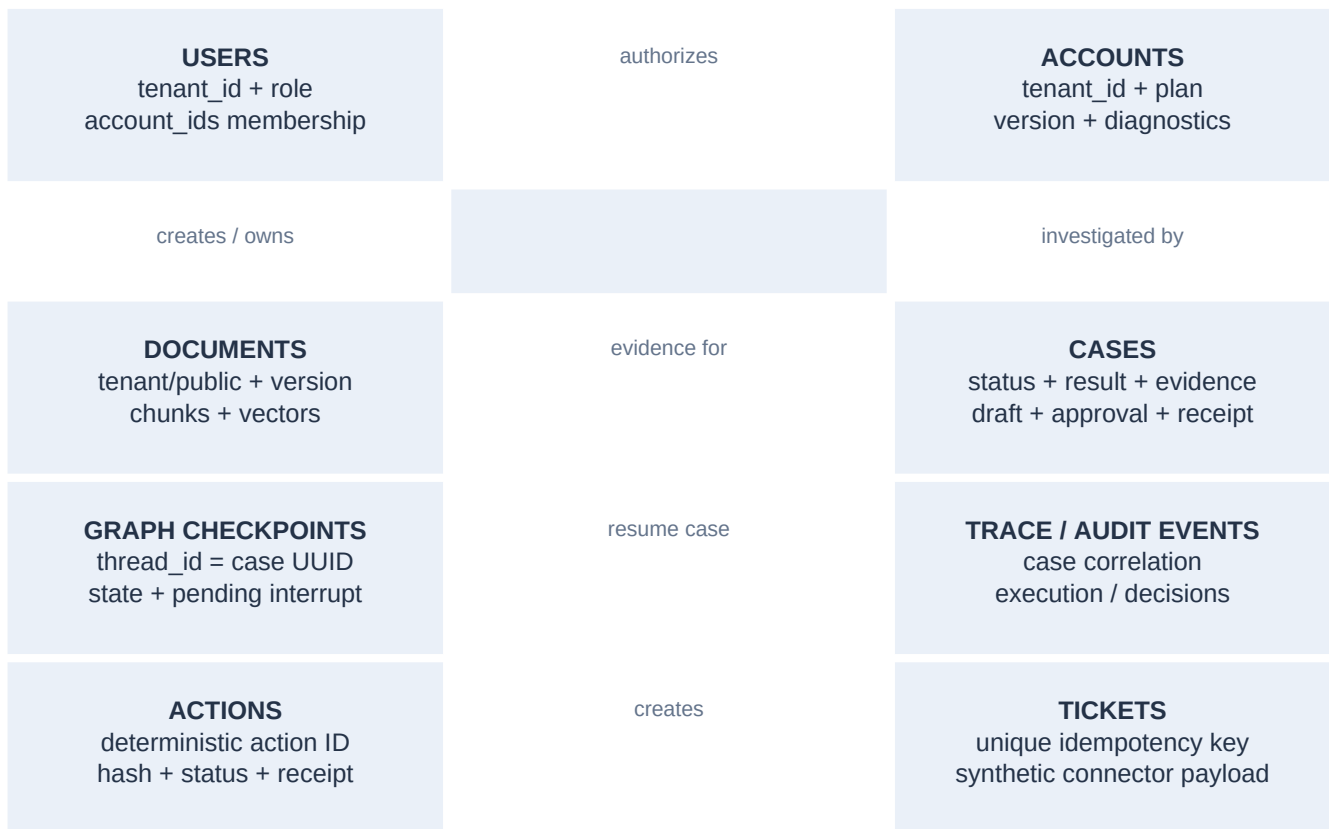


Figure 2. Domain relationship map. One account has many cases; each case can have many trace/audit events. A document can support many cases through evidence snapshots. An escalation uses a deterministic action key and a uniquely keyed ticket.

Storage decisions

SQLAlchemy stores operational entities in the application database. LangGraph stores execution checkpoints through its own SQLite or PostgreSQL checkpointer connection. They may share a database server, but are separate persistence concerns and are not a single atomic transaction.

There is no Tenant table in the current schema. Tenant IDs are carried on scoped rows; user account memberships are stored as JSON. Most identifiers are strings, and timestamps are epoch floats. Documents and escalation identifiers use deterministic naming/key schemes.

Production schema work / PROPOSED

Add explicit tenant/account/user relationships and migrations, normalize membership if its size or query needs grow, and define deletion behavior. Consider database row-level security as defense in depth. Do not infer these guarantees from the current SQLite foreign_keys setting alone.

Source: db.py; migrations/versions/*; workflow.py; actions.py

Data dictionary & lifecycle

Separate durable business facts, execution state and diagnostic telemetry.

Entity	Key fields and purpose
users / accounts	User ID, tenant, role, active flag and account membership; account plan, product version and synthetic diagnostic snapshot.
documents	ID, tenant, title, section, version, content/parent content, content hash, embedding JSON/vector, embedding version, active flag and update time.
cases	UUID, tenant/account/user IDs, redacted question, status/phase, result and evidence JSON, draft/approval/receipt JSON, safe error, lease, attempts and timestamps.
trace_events	Case ID, operation name, duration, status and redacted attributes. Optional trace persistence must not overwrite an execution failure.
audit_events	Tenant/case IDs, actor, action, details and time. Required for business decisions; ordinary SQL records, not a tamper-proof audit ledger.
actions / tickets	Deterministic action ID, payload hash, execution status and receipt; ticket idempotency_key has a unique constraint.
graph checkpoints	Serialized workflow state, node progress and interrupts. Case UUID is the graph thread ID; access remains mediated by the application.

Lifecycle and invariants

01	Create Store only the validated/redacted investigation request. Scope all later reads to the current principal and permitted account.
02	Investigate Preserve the exact evidence snapshots used to generate and verify claims. Reindexing a runbook does not rewrite old case evidence.
03	Approve and execute Bind approval to the exact draft hash and expiry. Revalidate actor, account snapshot and payload before producing a side effect.
04	Retain and erase / PROPOSED Set retention by record class. Erasure must cover case evidence, checkpoints, traces, audit policy exceptions, replicas and backup expiry.

Schema trade-off

JSON payloads allow evolving evidence and results without a migration for every model-output field. Pydantic validation is therefore part of the write contract. Promote frequently queried fields to indexed columns as operational queries emerge.

Source: db.py; schemas.py; security.py; actions.py

HTTP API surface

The console uses asynchronous case creation and explicit action approval.

Method + path	Contract / access
GET /api/health	Public process/configuration status. Not a dependency availability guarantee.
GET /api/ready	Public application DB check and embedded-worker liveness when enabled; 200 or 503.
GET /api/me	Authenticated server-derived identity, role, allowed accounts and mode.
GET /api/accounts	Authenticated account metadata filtered to tenant and membership.
POST /api/cases	Engineer only; account_id + question. Returns 202 with persisted queued case.
GET /api/cases	Scoped list; summary flag, limit 1-100, offset 0-10000. Default newest 100.
GET /api/cases/{id}	Scoped case detail: findings, evidence, draft, status and receipt.
GET /api/cases/{id}/trace	Scoped execution spans and audit events.
POST /api/cases/{id}/approval	Engineer only; exact payload_hash and strict approved boolean. State transition returns 202.
POST /api/cases/{id}/retry	Engineer only; eligible failed cases with processing attempts remaining. Returns 202.
GET /api/docs; /api/openapi.json	Public API reference/schema. No public ingestion or identity-provisioning API.
GET /; /assets/*	Built console HTML and hashed static assets.

Input and error contracts

Questions are trimmed and must contain 10-6,000 characters; account IDs are 1-64 characters. HTTP request bodies are limited to 32 KiB. Approval hashes must be 64 lowercase hexadecimal characters, and strings such as "true" are not accepted as booleans.

401 / 403 / 404

Invalid identity / forbidden role / missing or out-of-scope resource.

409 / 413 / 422

Conflicting state / oversized body / invalid schema or rejected input.

429 / 503

Tenant admission limit / dependency unavailable. Storage 503 responses include Retry-After: 5.

Source: api.py; schemas.py. Paths describe the current application, not a proposed public API gateway.

Request-to-result sequence

The request persists work quickly; investigation proceeds outside the HTTP response.

01	Browser -> POST /api/cases Attach identity, account_id and question. Disable duplicate submission while the request is pending.
02	API -> authorization + validation Resolve trusted membership, require engineer role, enforce body/rate limits, scan input and mask sensitive data.
03	API -> SQL -> browser Persist queued case and return HTTP 202. An accepted request is not a completed investigation.
04	Worker -> SQL lease -> LangGraph Atomically claim an eligible case; increment attempt count; start or resume the case thread.
05	Graph -> retrieval / tools / LLM Produce scoped evidence, structured findings and a verified result. Persist checkpoints at workflow boundaries.
06	Browser -> summary/detail reads Poll active work, then fetch changed selected-case detail. Render terminal result or approval request.
07	Engineer -> approval -> worker Review exact draft. Submit its hash and boolean decision. Worker resumes the interrupt, revalidates and executes only if approved.

Figure 3. End-to-end sequence. SQL admission, checkpoint writes and provider calls have distinct failure boundaries; a successful HTTP 202 does not imply provider success.

Illustrative requests

```
POST /api/cases
{"account_id":"<authorized-account-id>",
"question":"Investigate HTTP 429 sync failures. Check jobs,
incident status and entitlement. Do not escalate."}
```

```
POST /api/cases/{case-id}/approval
{"payload_hash":"<exact 64-character draft hash>",
"approved":true}
```

Client retry rule

GETs can be retried with bounded backoff. Mutations are not automatically retried by the console. If a creation response is lost, check existing cases before submitting again: POST /cases does not currently expose a client idempotency key.

Source: api.py; worker.py; frontend/src/{App,api,useRequestPolling}.ts*

The agent is a bounded state machine

LangGraph orchestrates specialized steps inside one investigation workflow.

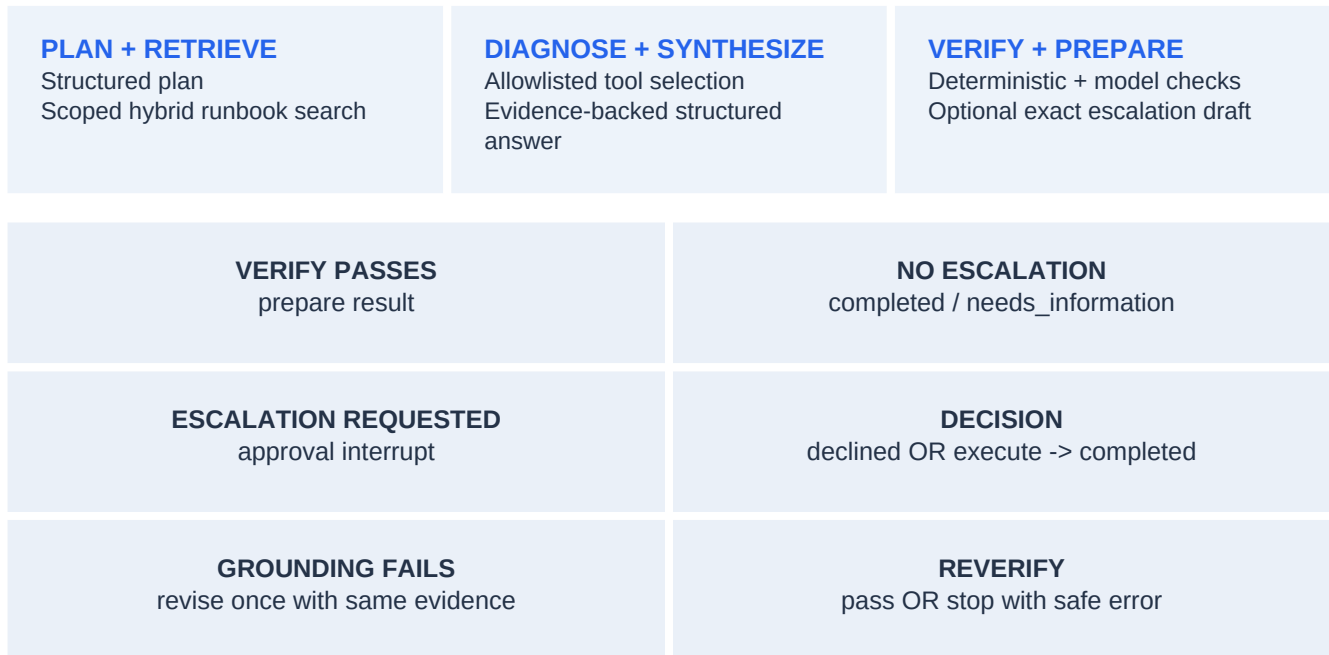


Figure 4. Workflow branch map. Provider policy refusals and sensitive/unsafe output do not enter the grounding-repair branch.

State and bounded autonomy

Graph state contains case_id, plan, evidence, result, verified, verification_feedback, revision_count, approved, receipt and status. Persistent checkpoints allow recovery and human approval across process restarts. The current principal/account is reloaded as work proceeds; checkpoint state is not authorization.

Diagnosis allows up to three model tool-selection rounds. Tool names and argument schemas are fixed; duplicate selections are removed. The configured tool-call ceiling is six, with four diagnostic tools currently registered. Scope mismatches and extra arguments are rejected. There is no arbitrary shell, SQL or URL-fetch tool.

Verification is a gate, not a score decoration

Structured output must pass schema checks and safety scanning. Each supported fact or hypothesis needs a contiguous excerpt from one cited evidence item; combining snippets into a fabricated quote fails. A separate live model call then judges semantic support. A valid quote proves provenance, not that the conclusion logically follows from it.

Failure behavior

One correction can use the same evidence and redacted feedback. A second failure stops the case and creates no escalation. The system does not claim that an LLM judge eliminates hallucinations.

Source: workflow.py; guardrails.py; schemas.py. Persistence rationale: [S1].

Retrieval & context engineering

Hybrid retrieval joins semantic meaning, exact operational terms and access control.

01	Ingest and version Split Markdown by headings, then approximately 350-word chunks with 40-word overlap. Store parent sections, content hashes and encoding fingerprints.
02	Encode locally / LOCAL CONFIG Pinned all-MiniLM-L6-v2 produces normalized 384-dimensional vectors on CPU. Long text uses overlapping token windows and normalized mean pooling.
03	Filter before ranking Restrict to public or principal-tenant documents, compatible product version, active rows and exact embedding version.
04	Rank and fuse PostgreSQL exact cosine search plus English full-text search; top 30 per ranker. Fuse reciprocal ranks, then deterministic semantic/lexical reranking.
05	Expand and compress Deduplicate by title/section; include parent context. Retrieve five initial items and up to two error-code expansion items. Preserve warnings and literal source excerpts.
06	Pack evidence budget Prioritize diagnostic snapshots. Tool JSON remains intact or is dropped; runbooks are extractively compressed to fit the configured evidence budget.

Figure 5. Retrieval pipeline. SQLite uses an eligible-document scan and bounded top-k selection; PostgreSQL uses pgvector and a full-text GIN index. No approximate vector index or trained cross-encoder is currently used.

Why embeddings matter here

An engineer may ask about "requests being throttled" while the runbook says "rate limit exceeded." Semantic vectors help retrieve that conceptual match. Full-text retrieval complements them for error codes, service names and exact identifiers. The old lexical-hash baseline is deterministic, but does not encode semantic meaning.

Reindexing contract

Changing model, revision, dimensions or pooling changes the embedding fingerprint. Reindex bundled documents and re-ingest custom documents; never compare vectors across versions. Existing case evidence remains an immutable historical snapshot in the application flow.

Source: retrieval.py; local_embeddings.py. Vector search trade-offs: [S2].

Approval & side-effect integrity

Human approval binds to a specific action payload, not a general permission to act.

01	Prepare the draft Server builds create_escalation with case/account IDs, account snapshot hash, support queue, summary and evidence IDs.
02	Bind and display Compute SHA-256 payload hash and expiry, default one hour. Display exact content for engineer review.
03	Record the decision Conditional transition from needs_approval to queued. Store actor, hash, decision time and expiry. A conflicting decision returns 409.
04	Reauthorize at execution Require active engineer membership and approved=true; verify payload hash, expiry and current account snapshot.
05	Execute transaction Synthetic ticket, action ledger, audit event and case receipt are written in one SQL transaction. Unique idempotency key blocks duplicate ticket insertion.
06	Return durable receipt Previously successful matching actions return their recorded receipt. Replayed workflow nodes must not assume a new side effect is needed.

What the current guarantee covers

The local synthetic action uses transactional SQL and deterministic identifiers. A concurrent race can still surface a uniqueness conflict that must be handled as an execution outcome. This is not a proof of exactly-once delivery to an external ticketing system.

Real connector design / PROPOSED

Write an outbox record in the same transaction as the accepted action. A connector worker sends a stable idempotency key to the remote service, records the receipt and reconciles ambiguous timeouts through a remote lookup. If the remote API cannot deduplicate or prove whether a request succeeded, hold the action for manual reconciliation rather than blindly retrying.

Stale approvals are invalid

A changed account plan, diagnostic snapshot, product version or expired draft can invalidate approval. The safe recovery is a fresh draft and new human decision, not silently mutating an already approved payload.

Source: actions.py; workflow.py; api.py; tests/test_workflow.py

Console & interaction contracts

Efficient reads and explicit status keep the UI aligned with asynchronous work.

Concern	Implemented behavior
Initial session	Fetch identity and scoped accounts on session changes. Role controls write buttons; backend repeats enforcement.
Case list	Fetch summary fields only. The console displays the latest 100; API pagination exists, but the UI has no load-more control.
Active polling	Sequential requests every two seconds while work is active. Pause in hidden tabs; error backoff grows to 30 seconds.
Case detail	Fetch when selection or the selected summary update timestamp changes. Unchanged summaries do not trigger full-detail fetches.
Traces	Load only in the trace view; poll active selected work and stop when terminal. Trace rendering is separated and memoized.
HTTP lifecycle	15-second timeout, AbortController cleanup and session-bound stale-response protection. No automatic mutation retry.
Evidence / errors	Safe JSON formatting, clipboard failure handling and clear safe-stop messages. A 401 returns to authentication.

Working states queued / running Show current phase; keep reads bounded.	Human decision needs_approval Inspect draft and choose approve/decline.	Terminal outcomes completed / needs_information / declined / failed Show result, missing facts or safe failure.
--	--	--

Measured optimization

On the recorded 11-case development dataset, list payloads fell from 55,176 bytes to 4,241 bytes with summary reads: a **92.3% reduction before compression**. This measures response size, not LLM latency, user-perceived speed at scale or production throughput.

Product boundaries There is no token streaming, case follow-up chat, full OIDC PKCE sign-in flow or push event channel today. A needs_information result is terminal; a user must start a new case or follow a manual process. Phase 2 should make that continuation explicit.
--

Source: frontend/src/*; docs/VERIFICATION.md (3 October review).

02 / Frameworks & storage

Choose a small operational footprint, with clear migration paths as demand grows.

Layer	Technology	Rationale / trade-off
Console	React 19 + TypeScript 5.8	Typed case/evidence contracts and reusable views. Client state remains small; no global state platform is required.
Build / UI	Vite 6 + CSS + Lucide	Fast development and static production assets. Visual components stay simple; API owns business rules.
HTTP / schema	Python 3.12-3.13; FastAPI; Pydantic 2	Python ecosystem fits inference and retrieval. Typed validation supplies API and model-output boundaries.
Persistence	SQLAlchemy 2 + Alembic	Shared domain access for SQLite and PostgreSQL. Explicit migrations replace schema creation in production.
Production database	PostgreSQL 17 + pgvector	Transactions, scoped metadata, full-text and vector retrieval in one database. Avoid a separate vector service until workload evidence warrants it.
Local database	SQLite WAL	Low-friction local setup and tests. Not the multi-replica production storage strategy.
Orchestration	LangGraph 1 + durable checkpoints	Explicit state transitions, bounded cycles, recovery and approval interrupts. More predictable than unconstrained agent loops.
LLM integration	LangChain OpenAI adapter + HTTPX	OpenAI-compatible structured output and native tool schemas. Compatibility must be tested per provider.

Why PostgreSQL over a dedicated vector database?

The primary workload is transactional case management with tenant-scoped retrieval over a modest runbook corpus. Keeping metadata and vectors together simplifies filtering, backups and migrations. A dedicated vector service becomes attractive only after corpus size, query load or isolation requirements exceed the measured PostgreSQL design.

Version policy

Technology families shown here reflect the repository, not a claim of newest releases. `uv.lock` and `frontend/package-lock.json` are the reproducible dependency authorities. Upgrade through tests, migration checks and evaluation gates.

Source: `pyproject.toml`; `frontend/package.json`; `Dockerfile`; `compose configuration`.

Inference, embeddings & guardrails

Use model capabilities selectively and keep budgets outside model control.

Capability	Implementation / rationale
Model routing	Separate small and reasoning routes, with complexity-aware selection. The local configuration maps both to the supplied gpt-6-luna endpoint; it is not currently multi-provider redundancy.
Structured results	Pydantic schemas passed through strict JSON-schema structured output for plans, findings, corrections and verdicts. Native tool binding for diagnostic selection.
Local semantic encoder	sentence-transformers/all-MiniLM-L6-v2; 384 dimensions, CPU, pinned revision, safetensors and no remote model code. Cached/offline loading supported.
Safety framework	Application checks plus optional Guardrails AI validation. Regex PII/injection checks and schema validators are useful layers, not complete DLP or jailbreak prevention.
Provider failover	Optional explicitly configured fallback route. Transient failures may retry/fail over; policy refusals and most configuration errors stop.
Budget enforcement	Default 4,000 completion tokens, 6,000 evidence budget, 16,000 configured model context and 45-second call timeout. Remaining workflow deadline bounds calls.

Routing and failure classification

A transient connection error, timeout, HTTP 408/429 or server error allows at most two attempts per route with short jitter. A route circuit opens for 30 seconds in process memory. A configured alternate route can then be considered. Schema failure gets one schema-repair instruction; repeated invalid output stops rather than switching providers blindly.

Known provider policy blocks are refusals: no fallback and no retry. Output truncation is reported explicitly. Default workflow deadline is 90 seconds; the local live setup has used 150 seconds. Byte-based prompt reserves are conservative estimates, not exact model-token accounting.

Local embedding privacy boundary

Embedding text stays on the local encoder. Selected evidence still leaves the application for a live LLM request. Provider residency, retention, allowed destinations and data-processing terms must be approved for production.

Source: config.py; models.py; local_embeddings.py; guardrails.py.

Delivery, observability & evaluations

Operational visibility and answer quality need separate acceptance criteria.

Delivery

Docker multi-stage build
Node 22 -> Python 3.12 runtime
Non-root application user

Observability

Local trace/audit records
OpenTelemetry spans/export
Optional Phoenix deployment

Quality

pytest + Vitest + Playwright
Development eval scenarios
Optional Ragas metrics

Trace and cost contract

Record operation names, duration, route/model, token usage, configured cost estimates, tool calls, evidence identifiers and safe failure categories. Missing prices must be labeled unpriced. Redact content before persistence/export. The case correlation ID is not automatically proof of end-to-end OpenTelemetry trace-context propagation; verify propagation when services are separated.

Evaluation pipeline

01

Deterministic gates / IMPLEMENTED

Schema, authorization, injection, citation provenance, approval integrity, recovery and retrieval regression tests. Twelve curated development scenarios exercise expected behavior.

02

Semantic harness / AVAILABLE

Optional Ragas integration for faithfulness/factual correctness. A live held-out quality benchmark has not been reported.

03

Release benchmark / PROPOSED

Version a held-out dataset by issue family, tenant boundary and adversarial pattern. Track retrieval recall, factual accuracy, unsupported-claim rate, false safety blocks, latency and cost.

04

Decision / PROPOSED

Compare candidate and baseline on the same evidence and model versions. Require human review of regressions; never promote solely because average judge score rises.

Recorded verification, not rerun for this TDD

The 3 October review records 73 backend tests with 94% statement coverage, 13 frontend tests and 12/12 development eval scenarios. Dependency audits reported no known vulnerabilities at that time. Docker execution and external OIDC deployment remain unverified; these results are not a production SLA or security certification.

Source: [telemetry.py](#); [evals/*](#); [backend/tests/*](#); [frontend/e2e/*](#); [.github/workflows/*](#); [docs/VERIFICATION.md](#).

03 / MVP capabilities

A focused investigation product with reviewable evidence and controlled actions.

Feature	User value	MVP acceptance behavior
Account-scoped investigation	Analyze the right customer without mixing tenants.	Engineer selects a permitted account; an out-of-scope ID reveals no case data.
Diagnostic tool use	Correlate failed jobs, permissions, incidents and support coverage.	Only allowlisted read tools run; output becomes source evidence.
Semantic runbook retrieval	Find guidance even when wording differs.	MiniLM plus lexical retrieval, product-version filtering and cited excerpts.
Verified findings	Explain what is known and what remains uncertain.	Reject invalid citations; live semantic verification gates completion.
Evidence and execution trace	Let an engineer inspect how the answer was produced.	View diagnostic snapshots, runbook evidence, execution timing and safe failures.
Escalation draft + approval	Avoid silent changes to external operational workflows.	Show exact draft; require engineer approval bound to its hash and expiry.
Recovery and retry	Survive process interruption without uncontrolled action replay.	Persist graph state, use leases and offer retry only for eligible failures.
Read-only access	Allow observers to inspect investigations safely.	Viewer cannot create, retry or approve through UI or API.

Representative demonstration scenarios

Acme / 403

Check failed sync jobs and missing credential scope after rotation; verify premium entitlement.

Atlas / 429

Investigate throttling, check incident status and standard support; recommend cited recovery steps.

Meridian / 503

Compare sync failures with the matching regional service incident; prepare a reviewable escalation.

Data realism boundary

These accounts, diagnostics and tickets are synthetic adapters. Live mode changes LLM inference; it does not turn the connector data into real customer telemetry. Showcase this honestly in demos and interviews.

Source: seed.py; tools.py; workflow.py; frontend/src/App.tsx.

Phase 2 product roadmap

Prioritize production access and trustworthy actions before broadening autonomy.

Priority / enhancement	Design scope	Acceptance gate
P0 / Enterprise sign-in	OIDC browser authorization code + PKCE, session renewal and managed account provisioning.	External identity deployment tested; roles come from trusted application membership.
P0 / Real support connectors	Read adapters for telemetry and account systems; ticket connector with outbox and reconciliation.	Ambiguous remote writes cannot create uncontrolled duplicates; credentials are scoped.
P0 / Operations foundation	Separate worker endpoint, managed PostgreSQL, backups, SLOs, shared admission controls.	Failover and restore drills pass; capacity measured with realistic workload.
P1 / Knowledge lifecycle	Ingestion UI, source ownership, review dates, versioned replacement and revocation.	ACL tests pass; stale/revoked sources disappear from new retrieval results.
P1 / Case continuation	Collect missing evidence and resume a case with a new evidence version.	Old findings stay auditable; changed action payload always needs a new approval.
P1 / Push updates + history	SSE for case state events, pagination and search; accessible keyboard flows.	Reconnect reconciles from server state; no authorization leakage across subscriptions.
P1 / Quality operations	Held-out evals, reviewer feedback, regression slices and model canaries.	Quality thresholds and human adjudication are agreed before rollout.
P2 / Retrieval scale	ANN indexes, optional reranker and asynchronous ingestion.	Measured gain in latency or recall; tenant-filtered recall stays acceptable.

Deliberate non-goals for the MVP

No autonomous credential rotation, arbitrary remediation commands, general-purpose web browsing, unrestricted SQL, unsupervised ticket submission or promise of root-cause certainty. A regional incident match can support a hypothesis; it does not establish causation without adequate evidence.

Product success metrics / PROPOSED

Measure median investigation time, engineer acceptance/edit rate, evidence usefulness, escalation preparation time and avoided duplicate work. Pair them with false-claim rate, false rejection rate and unauthorized-action count. Establish baselines before claiming improvement.

Roadmap is a design proposal derived from current product boundaries; it is not shipped scope.

04 / Horizontal scaling strategy

Scale execution independently while preserving one authoritative case state.

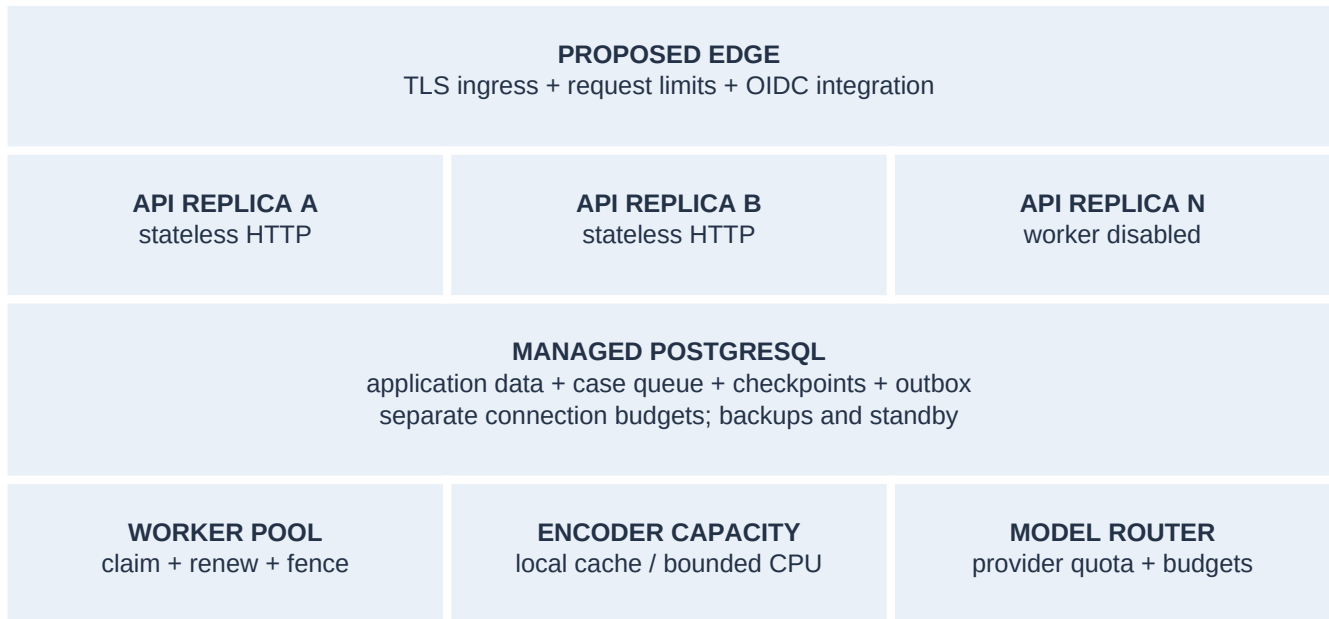


Figure 6. Target deployment. A standalone worker launcher, renewable leases, fencing and shared quota coordination must be implemented. The current application embeds one worker thread per API process.

Capacity planning with explicit assumptions

Illustrative sizing, not a benchmark: 12 investigations/minute and 40 seconds average machine time imply about 8 concurrent investigations (arrival rate x service time). At 60% planned utilization, provision roughly 14 worker slots, then validate CPU, DB connections and provider quotas under load. Human approval wait is persisted and should not occupy a worker slot.

Autoscale workers on queued work age and processing demand, bounded by model RPM/TPM and database connection capacity. Use a shared pool budget across API, worker and checkpoint connections. Keep approval/execution and authorization reads on the primary to avoid replica-lag decisions.

Retrieval scaling

Start with exact filtered search and measure p95 latency as corpus size grows. Introduce version/dimension-specific HNSW indexes only after testing tenant-filtered recall. Avoid a global approximate index that retrieves too few authorized results after filtering. Preload encoder weights and bound concurrent CPU inference. [S2]

Current: worker.py; db.py; retrieval.py. Target topology and sizing assumptions are proposed.

Failure, recovery & failover

Retry only when the outcome and safety boundary are understood.

Failure	Current behavior	Production extension
Transient model failure	Bounded attempts, jitter, per-process circuit and optional configured fallback.	Shared provider-health view, quota-aware backoff and tested approved failover routes.
Provider policy rejection	Stop with safe refusal classification; no retry or fallback.	Maintain policy compatibility across approved providers; alert on unexpected rejection spikes.
Invalid / unsupported answer	One grounding correction, then stop. No escalation on failure.	Human review queue and quality regression triage; retain false-block evidence.
Worker process loss	Persisted checkpoints; recover queued or expired running lease, up to four processing claims.	Lease heartbeat, fencing token, explicit worker deployment and crash/replay stress tests.
Application DB outage	Controlled 503; readiness fails on DB failure. Required persistence cannot be skipped.	Managed standby, bounded reconnect and tested point-in-time recovery.
Checkpoint DB outage	Persistent graph operations fail; application readiness does not separately probe this dependency.	Add checkpoint capability check without a costly full LLM health probe.
Remote action timeout	Current ticket adapter is synthetic SQL; no real remote ambiguity handling.	Outbox, stable idempotency key, remote reconciliation; unknown outcome needs operator review.

Lease semantics and limitations

Current lease duration is max(300 seconds, three times the configured workflow deadline). Claims increment attempts, including approval resumption. Recovery is bounded to four processing claims. There is no heartbeat or fencing token today, so slow/stalled workers and concurrent recovery require explicit hardening before scale-out.

Recovery objectives / PROPOSED

Initial targets: operational RPO $\leq$ 5 minutes and RTO $\leq$ 30 minutes, subject to business approval and restore drills. Reconcile cases, checkpoints and external action receipts after restore; backups of separate stores can represent different moments in time.

Source: worker.py; models.py; api.py; actions.py. RPO/RTO targets are not measured guarantees.

Rate limits, budgets & service objectives

Control admission, concurrency and inference spending as separate resources.

ADMISSION / CURRENT

20 new cases per tenant per minute
DB-coordinated check

EXECUTION / CURRENT

Bounded graph rounds and claims
Workflow + call deadlines

COST / CURRENT

Context/output budgets
Token usage + price estimates

Existing controls

PostgreSQL admission uses an advisory transaction lock; SQLite uses BEGIN IMMEDIATE to serialize the check. This prevents a simple multi-request race in the tenant creation quota. It does not limit per-IP abuse, concurrent model calls, active work per tenant or shared provider tokens per minute.

Context packing reserves output and instruction/schema space before evidence selection. Tool-call rounds, one correction, provider attempts and processing claims are all bounded. The frontend stops unnecessary polling; its 15-second HTTP timeout is independent of the longer background investigation deadline.

Production controls / PROPOSED

Control	Policy
Edge request quota	Per identity/IP burst protection and body-size enforcement; exempt only justified health checks.
Tenant fairness	Cap queued and active cases per tenant. Use a fair scheduler so one customer cannot monopolize workers.
Provider budget	Shared RPM/TPM semaphore/token bucket, maximum queued wait and backoff that respects provider signals.
Spend ceiling	Per-case and per-tenant daily limits; reserve estimated budget before dispatch and reconcile actual usage afterward.
Overload response	Reject or defer before costly inference; report Retry-After where meaningful. Never hide overload as a successful answer.

Candidate service objectives / PROPOSED

Start with monthly 99.9% authenticated read/admission availability, p95 create acknowledgment below 500 ms at an agreed load, and p95 machine processing below 90 seconds for routine cases. Exclude human approval wait from machine latency. Report completion quality separately: a fast unsafe answer is a failure, and a safe rejection still affects usefulness.

Source: config.py; api.py; models.py; workflow.py. SLO numbers are initial planning targets, not current performance claims.

Trust boundaries & secure execution

Treat model outputs, retrieved text and tool content as untrusted data.

IDENTITY BOUNDARY JWT verification -> stored user membership	DATA BOUNDARY tenant/account/version filters -> evidence
MODEL BOUNDARY schemas + fixed tools + bounded context	ACTION BOUNDARY exact approval + reauthorization + ledger

Figure 7. Independent enforcement layers. No model-generated text can grant permissions or turn an unapproved draft into an authorized write.

Threat	Implemented control	Residual risk / next step
Tenant data exposure	Server-side tenant/account checks on cases, tools and retrieval. JWT role claims do not override stored membership.	Add RLS and adversarial isolation tests across all new connectors and caches.
Identity spoofing	OIDC RS256 with issuer/audience and required exp/iat/sub; network errors -> 503, invalid token -> 401.	Demo headers are local-only trust. Deploy/test enterprise sign-in before production.
Prompt injection	Input and source pattern checks, constrained tool schemas and output checks. No arbitrary SQL/shell tool.	Patterns are incomplete. Test indirect injection in runbooks/tool output; enforce policy outside prompts. [S3]
Unsupported claims	Exact contiguous citation provenance plus independent semantic judge; one correction only.	Judge can be wrong. Maintain held-out evals and human escalation path.
Unauthorized action	Engineer role, exact payload hash, expiry, account snapshot check and idempotency ledger.	Remote connector semantics still require outbox and timeout reconciliation.
Secret / PII exposure	Mask common PII/secrets; suppress sensitive config representation; redact safe errors/traces.	Not comprehensive DLP. Minimize source data, use managed secrets and test export redaction.

Defense in depth

Prompt instructions are not an access-control mechanism. A model may misread malicious content, but authorization, fixed tool dispatch, evidence verification and approval checks must still prevent unauthorized side effects. [S3]

Source: security.py; guardrails.py; tools.py; actions.py; config.py.

Data protection & deployment gates

Production configuration checks are necessary; they do not provision secure infrastructure.

Implemented safeguards

Production settings require OIDC, disabled demo seeding/automatic schema creation, PostgreSQL for application and checkpoint storage, HTTPS identity/model endpoints and explicit allowed hosts. Container configuration uses a non-root user, drops capabilities and enables no-new-privileges. Default local published ports bind to loopback.

HTTP headers include nosniff, no-referrer, frame denial and a restrictive content security policy with specific allowances for inline styles/API documentation. API/HTML responses are not cached; hashed static assets can be cached for one year. Environment variants are excluded from the Docker build context.

Deployment checklist / PROPOSED

Gate	Required evidence
Secrets and provider policy	Rotate the previously shared development credential before production. Inject secrets through a managed store; approve provider destinations, data retention and fallback routes.
Encryption and network	Terminate TLS at ingress; verify database TLS and at-rest encryption. Restrict egress and private database access. Configure exact trusted hosts.
Database change control	Apply and verify Alembic head in a release job; back up first. Startup checks for a version table are not sufficient proof of schema currency.
Data lifecycle	Define retention for cases, evidence, checkpoints, traces, audits and backups. Implement deletion and access/export procedures; no automatic retention exists today.
Identity and demo cleanup	Provision least-privilege memberships and test revocation. Disabling seeding does not remove previously seeded demo records.
Supply chain	Pin production images, scan dependencies/artifacts, produce an SBOM and sign releases. The optional Phoenix latest tag is not a production pin.
Runtime operations	Exercise container startup, external OIDC, failover, restore, quota saturation and graceful shutdown in staging.

Go-live decision

Use a limited tenant pilot only after these gates and the held-out quality benchmark pass. Keep action approvals mandatory, define an incident owner and make model/action kill switches operationally accessible. This document does not certify regulatory compliance.

Source: config.py; api.py; Dockerfile; .dockerignore; docs/OPERATIONS.md and VERIFICATION.md.

Decisions, traceability & release evidence

Use this page as the handoff between architecture, implementation and operations.

Decision	Why / revisit trigger
Modular monolith first	Keeps transaction and policy boundaries understandable. Split worker deployment when execution load or failure isolation demands it.
PostgreSQL + pgvector	Combines relational state and scoped hybrid search. Revisit when measured corpus/query requirements justify specialization.
Local MiniLM encoder	Removes the provider embedding-access dependency and supports semantic search. Revisit quality/dimensions with a versioned retrieval benchmark.
Fail closed on verification	Avoids presenting unsupported findings or creating unjustified actions. Track false rejection rate and provide a human review path.
Exact human approval	Makes side effects intentional and reviewable. Extend autonomy only for narrowly scoped, reversible actions with separate authorization.

Source map

System: backend/supportops/{api,db,schemas,worker,workflow,actions}.py.

AI and safety: {models,retrieval,local_embeddings,tools,guardrails,security}.py.

Console: frontend/src/{App,api,useRequestPolling,TraceView,types,format}.ts*.

Delivery: pyproject.toml, lockfiles, Dockerfile, migrations, CI workflows.

Verification: docs/VERIFICATION.md; backend/tests; frontend tests; evals.

External design references

[S1] **LangGraph persistence.** Checkpoint/thread semantics and durable state.

docs.langchain.com/oss/python/langgraph/persistence

[S2] **pgvector documentation.** Exact/approximate search, indexing and filtering trade-offs.

github.com/pgvector/pgvector

[S3] **OWASP LLM Prompt Injection Prevention Cheat Sheet.** Defense-in-depth guidance.

[cheatsheetseries.owasp.org / LLM Prompt Injection Prevention](https://cheatsheetseries.owasp.org/LLM-Prompt-Injection-Prevention)

Document assurance

Source-based architecture review dated 3 October 2026, revision 603a7da. Historical test results are attributed to the repository verification record; they were not rerun to create this PDF. Proposed capacity, SLO and recovery targets require measurement and owner approval. No application credentials are included.